Claude Code

A Hands-On Primer

Derrick Bozkurt · *Prompt Engineers*

Follow along → github.com/debozkurt/claude-intro

Agenda

#	Section
0	Mental model — what Claude Code *is*
1	Install & first run
2	Ecosystem & structure
3	Core usage & control
4	Extensibility primitives
5	Live build walkthrough
6	Q&A, links, take-home

0 · Mental model

What Claude Code actually is

An **agentic coding CLI**. Lives in your terminal, reads/edits files in your codebase, runs shell commands, completes multi-step tasks — not just autocomplete or chat.

Landscape around it:

- **GitHub Copilot** — editor-native autocomplete; now has an agent mode too.
- **Cursor / Windsurf** — IDE-native agents. Editor is the home; multi-file chat + edits baked in.
- **OpenAI Codex CLI** — closest peer to Claude Code. Terminal-native agent, different model family.
- **ChatGPT / Claude.ai (web)** — chat only. No direct file or shell access to your repo.

Claude Code — terminal-native, deeply integrated with the Claude family, extensible via skills / subagents / hooks.

Why this matters for how you prompt it

Because it has **file + shell access**, you can describe outcomes, not implementations:

- ❌ Copilot-brain: *"Write a function that..."*
- ✅ Claude Code: *"Add pagination to the user list endpoint. Check how other endpoints do it first."*

It'll `Read` similar files, follow your patterns, edit, and (if permitted) run tests.

The rest of this talk: how to set it up. The setup is what decides whether Claude Code stays a novelty or becomes part of your daily workflow.

1 · Install & first run

Prerequisites

Following along live? Install Claude Code and log in **before** we start — takes ~5 min (next slide).

What you need:

- **git** (`git --version`)
- **macOS, Linux, or Windows + WSL**
- A Claude account (claude.com) or an `ANTHROPIC_API_KEY`

Install

macOS / Linux / WSL

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows (PowerShell)

```
irm https://claude.ai/install.ps1 | iex
```

Homebrew (macOS)

```
brew install --cask claude-code
```

Full guide → docs.claude.com/en/docs/claude-code/quickstart

Verify: `claude --version`

First run

```
cd ~/your-project     # or any directory
claude                # launches the interactive TUI
```

Inside the session, run `/login` — opens a browser, authenticates, drops a token in `~/.claude/`.

Authentication options

Option 1 — `/login` (recommended)

Uses your Claude subscription (Pro / Max / Team). Zero extra config.

Option 2 — API key (pay-per-token)

```
export ANTHROPIC_API_KEY=sk-ant-...
claude
```

Which one? Pro / Max is flat-rate — great for steady daily use. API key makes sense for automation, team billing, or if you're already an Anthropic API customer.

What you see in a session

- **Input box** at the bottom — type your prompt, `Enter` to send.
- **Streaming output** above — Claude's reasoning, tool calls, results.
- **Shift+Tab** — cycles permission/mode (default ↔ plan ↔ accept-edits).
- **Esc** — interrupts the current turn. Press twice = rewind (more in §3).
- `/help` — lists every slash command available.

Starting Claude — the flags you'll actually use

Flag	What it does
`claude`	Interactive TUI in current dir
`claude -p "task"`	Headless — one-shot, prints to stdout, exits
`claude --continue` (`-c`)	Resume most recent session in this dir
`claude --resume`	Pick a session from recent history
`claude --model <name>`	Start with `opus` / `sonnet` / `haiku`
`claude --permission-mode <mode>`	Start in `plan` / `acceptEdits` / `default`
`claude --add-dir <path>`	Include another dir in working scope
`claude --dangerously-skip-permissions`	Bypass ALL prompts — sandbox only
`claude --version`	Print version

Full reference (plus keyboard shortcuts) in `cheatsheets/commands.md`.

Our demo target — `lab-app/`

A slim Claude-themed landing page for *this* session lives in the repo at `lab-app/`:

- Plain HTML + CSS — no build step
- Has a real `CLAUDE.md` with conventions and guardrails
- Has a pre-configured `.claude/settings.json`
- Small enough that edits are visible instantly

Throughout the talk, ▶️ **DEMO** callouts mean: *we'll run this live against* `lab-app/`. Follow along if you've cloned the repo, or just watch.

```
cp -r lab-app ~/scratch/claude-lab
cd ~/scratch/claude-lab && open index.html
claude
```

2 · Ecosystem & structure

Global vs project config · CLAUDE.md precedence · @path imports · memory model

The two homes

Claude Code reads config from two places, merged at session start:

Location	Scope	Commit?
`~/.claude/`	**Global** — applies to every project you run	**Yes** — in a personal dotfiles repo
`<repo>/.claude/`	**Project** — this repo only	**Yes** — in the project repo

Plus: `<repo>/.claude/settings.local.json` = your personal overrides for this project (gitignored).

Committing `~/.claude/` is a superpower — restore your full Claude setup on any machine. But be deliberate about what you share vs. what stays local (next slide).

Sensible gitignore for `~/.claude/`

Whitelist what you share; exclude state, logs, credentials:

```
# ~/.claude/.gitignore

# state + ephemeral — personal and large
projects/              # per-session history
todos/
statsig/
logs/
*.log

# credentials
.credentials.json
*.token

# caches
cache/
ide/
```

Commit: `CLAUDE.md`, `settings.json`, `skills/`, `agents/`, `commands/`, `hooks/`, `keybindings.json`.

Inside `~/.claude/` (global)

```
~/.claude/
├── CLAUDE.md           # instructions loaded into EVERY session
├── settings.json       # permissions, env, hooks, model prefs
├── commands/           # custom /slash-commands (personal)
├── skills/             # skills (personal)
├── agents/             # subagents (personal)
├── projects/           # per-project session history + memory
└── keybindings.json    # custom terminal keybinds
```

Think of it like `~/.ssh/` or `~/.gitconfig` — your personal Claude Code profile, richer.

Directories worth knowing — config

Committed — you author these, safe to share in dotfiles:

Dir	What lives here
`skills/`	Skills (SKILL.md + resources) invoked by `/name`
`agents/`	Subagent definitions (prompt + tool allowlist)
`commands/`	Custom `/slash-commands`
`plugins/`	Installed plugins (skills/agents/commands bundled)

Plus `CLAUDE.md`, `settings.json`, `keybindings.json` at the root.

Directories worth knowing — runtime state

Gitignored — Claude writes these as you work:

Dir	What lives here
`projects/`	Per-project session transcripts **+** `memory/` **files**
`plans/`	Saved plan-mode outputs
`todos/`	In-session task lists
`paste-cache/`	Images/large text pasted into the TUI
`shell-snapshots/`, `session-env/`	Captured shell state per session
`file-history/`	Pre-edit snapshots (powers `/rewind`)

Why it matters: `projects/<slug>/memory/` is where auto-memory persists. Back it up, grep it, or wipe a slug to reset what Claude "knows" about a repo.

Inside `<repo>/.claude/` (project)

```
your-repo/
├── CLAUDE.md                    # project-wide instructions
├── .claude/
│   ├── settings.json            # team-shared permissions/env
│   ├── settings.local.json      # personal overrides (gitignored)
│   ├── commands/                # project slash commands
│   ├── skills/                  # project skills
│   └── agents/                  # project subagents
└── subfolder/
    └── CLAUDE.md                # subfolder-scoped instructions
```

CLAUDE.md hierarchy & precedence

Claude walks up from your working directory and loads every `CLAUDE.md` it finds. **All concatenated** — more specific wins by being loaded later.

Scope	Location	Who it's for
User	`~/.claude/CLAUDE.md`	Your preferences, all projects
Project	`./CLAUDE.md` or `./.claude/CLAUDE.md`	Team, committed
Subfolder	`<repo>/sub/CLAUDE.md`	Loaded on-demand when Claude touches that dir

Why it matters: global *preferences* (your style) at user level. Project *facts* (stack, conventions) at project level. *Hyper-local rules* (one folder uses a different lint config) at subfolder level. Claude stitches them together for you.

What actually gets sent to the model

On each turn, Claude Code assembles a prompt:

```
[ Claude Code system prompt (built-in) ]
+ [ Your global CLAUDE.md ]
+ [ Project CLAUDE.md ]
+ [ Relevant subfolder CLAUDE.md (if you're in/touching that folder) ]
+ [ Tool definitions: Read, Edit, Bash, Grep, Glob, ... ]
+ [ Conversation so far ]
+ [ Your latest message ]
```

Your codebase is NOT sent. Files are read **on demand** via the `Read` tool. This is why Claude Code works on huge repos — it only pulls what it needs.

`@path` imports inside CLAUDE.md

Pull in existing docs instead of rewriting them. Real example from our lab app — `lab-app/CLAUDE.md`:

```
- Visual style deliberately mirrors the slide deck. See the full palette,
  typography, and contrast rules in @docs/design-tokens.md — reuse the
  tokens defined there instead of hardcoding colors.
```

At session start, `@docs/design-tokens.md` is **expanded inline** — Claude sees the full contents (palette table, typography stack, contrast rules) as if you'd pasted them directly into `CLAUDE.md`. Up to **5 levels** of recursive imports.

▶️ **Try it:** ask Claude `what's the accent color?` — it answers `#d97757` with zero tool calls, because the token doc was pre-loaded via the import.

Why it matters: CLAUDE.md becomes a *table of contents*, not a re-documentation. Keep it short; hoist the rest via `@path`. Designers can edit `design-tokens.md` without touching Claude config.

`.claude/rules/` — modular, scoped instructions

Topic-specific files Claude auto-loads alongside `CLAUDE.md` — **always-on** or **path-scoped**:

```
lab-app/.claude/rules/
├── no-build-tooling.md        # always-on - no frontmatter
└── html-accessibility.md      # path-scoped - loads only on *.html
```

Frontmatter gates path-scoped rules:

```
---
paths:
  - "**/*.html"
---
# HTML accessibility
Every <img> needs alt text. Use semantic tags. WCAG AA contrast...
```

▶️ **Try it:** ask Claude to edit `styles.css` — the HTML rule stays out of context. Ask it to edit `index.html` — the accessibility rule loads automatically.

Why it matters: `CLAUDE.md` stays lean; specialized rules only load when relevant. Symlinking `~/.claude/rules/` across repos kills copy-paste drift.

`settings.json` precedence

Same three-layer idea, different merge:

```
~/.claude/settings.json              ← user defaults
<repo>/.claude/settings.json         ← team settings (committed)
<repo>/.claude/settings.local.json   ← your personal overrides (gitignored)
```

Merge rule: later layer wins for the same key. Permissions arrays (`allow` / `deny` / `ask`) are **unioned** — deny from any layer wins.

Auto memory — Claude writes its own notes

On by default. Claude captures learnings (your corrections, preferences, build quirks) into a memory directory **nested per project** (not at `~/.claude/` root):

```
~/.claude/projects/<project-slug>/memory/
├── MEMORY.md              # concise index – loaded every session
├── debugging.md           # on-demand topic file
└── api-conventions.md     # on-demand topic file
```

- `<project-slug>` is the absolute project path with `/` → `-`. E.g. `~/Downloads` becomes `-Users-you-Downloads`.
- `MEMORY.md` loads at session start — first **200 lines** or **25KB**, whichever comes first.
- **Topic files** are pulled on demand — no context cost until Claude actually needs them.
- Keyed by git repo → all worktrees of the same repo share one memory dir.
- **Machine-local.** Not synced across machines or CI.

Turn off with `CLAUDE_CODE_DISABLE_AUTO_MEMORY=1` or `{"autoMemoryEnabled": false}` in settings.

Managing memory actively

When to reach for what:

Want to...	Use
Give Claude a *rule* you'll enforce	Write it into `CLAUDE.md`
Scope a rule to specific files	A path-scoped rule in `.claude/rules/`
Let Claude remember *without* you writing	Auto memory (on by default)
Pull in an existing doc as context	`@path` import
Review/edit what's loaded this session	`/memory`
Compress a long session	`/compact`
Wipe this session, keep CLAUDE.md	`/clear`

Rules of thumb:

- CLAUDE.md under **200 lines** — split into rules or imports beyond that.
- Run `/memory` periodically to audit what Claude has learned about you.
- If `/compact` is losing key instructions, move them into CLAUDE.md instead of re-typing.

▶️ Try it — write your first CLAUDE.md

Drop this into any repo. Focus on *context Claude can't infer* and *rules that prevent mistakes*:

```
# Project: <your-project>

## Context
- What this project does, in one paragraph.
- Who the users are and the primary flow.
- Any non-obvious architectural decisions.

## Conventions
- How tests are organized and how to run them.
- Naming / style rules your team actually enforces.
- What "done" looks like for a change in this repo.

## Guardrails
- Directories NOT to modify (legacy, generated, third-party).
- Dependencies NOT to add without discussion.
- Destructive operations that require human approval first.
```

Now `claude` picks this up every session. Ask it to ship a change — watch the guardrails hold without re-telling.

3 · Core usage & control

Plan mode · headless · worktrees · permissions (allow/deny/ask) · ultrathink

Essential slash commands

Command	What it does
`/help`	Lists every slash command
`/login` / `/logout`	Auth
`/model`	Switch model (Opus ↔️ Sonnet ↔️ Haiku)
`/memory`	Browse / edit loaded CLAUDE.md + auto-memory files
`/permissions`	Manage allow / deny / ask rules
`/compact` / `/clear`	Context management
`/agents`	Manage subagents
`/hooks`	Manage hooks
`/config`	Open settings
`/cost`	Session token + cost usage

Type `/` at any time to see the full list in your session.

Running modes vs permission modes

Two distinct axes:

Running mode — *how* you invoke Claude:

- **Interactive** — `claude` launches the TUI.
- **Headless** — `claude -p "task"` runs once, prints output, exits.

Permission mode — *what* Claude is allowed to do without asking:

- **Normal** (default) — asks before every tool call.
- **Auto-accept edits** — approves `Read` / `Edit` / `Write`; still asks for `Bash` etc.
- **Plan** — read-only; plans but doesn't edit or run state-changing commands.
- **Bypass permissions** — skips every prompt. Sandbox only. (CLI flag; not in the cycle.)

Cycle **Normal → Auto-accept edits → Plan** with `Shift+Tab` inside an interactive session. Details on each below.

Plan mode

```
# cycle to plan mode with Shift+Tab, then:
> Refactor the auth middleware to use JWT instead of session cookies.
```

Claude:

1. Reads the relevant files (auth.js, routes, tests)
2. Produces a written plan — files to touch, order, risks
3. **Does not edit** until you approve

Why it matters: plan mode is your dry-run. Non-trivial changes should start here — catches misunderstandings before Claude writes 200 lines in the wrong place.

▶️ **DEMO** · In `lab-app/`, `Shift+Tab` to plan, then: *"Add a 'Session agenda' section to `index.html` between Audience and Prereqs — 6 rows (time · topic) matching the slide-deck agenda. Use the existing `<section>` + `<h2>` pattern and the coral palette."* Watch the plan appear — cross-reading CLAUDE.md + `styles.css` — before a line is written.

Headless mode

```
claude -p "list every TODO comment in src/ with file:line"
claude -p "summarize the last 10 commits" > release-notes.md
claude -p "does this repo have a CI config? which CI?" | grep -i yes
```

Use cases:

- Scripts and automation · CI pipelines · piping into Unix tools (`| jq` , `| grep`)
- **Scheduled / remote execution** — cron jobs, remote triggers, Slack/GitHub webhooks firing Claude

Why it matters: headless is the foundation for **remote control** — running Claude without a human at the terminal. *Remote agents and scheduling could fill their own meetup — scope for next time.*

▶️ **DEMO** · In a second terminal: `claude -p "list every section heading in lab-app/index.html"` . No TUI, just stdout. Then pipe it: `... | wc -l` .

Session control — resume, rewind, fork

Three ways to navigate a conversation. Pick the right one:

Problem	Tool
Closed the terminal; want to pick up where you left off	**Resume** — `claude --continue` (last session) or `claude --resume` (picker)
Wrong last turn; want to retry from a checkpoint	**Rewind** — `Esc` - `Esc` inside the session, pick a turn
Want to try a different *conversation* path without losing this one	`/branch` — forks the transcript only; same dir, same branch, same files
Want an isolated *codebase* copy to experiment in	**Worktree** — `git worktree add ../scratch -b exp`, then launch `claude` there (new dir → new session automatically)

Why this matters: conversations are cheap; discarding context is expensive. `/branch` when you want to retry the *chat*; worktree when you want to retry the *code*. Don't just `/clear`.

Context management

Long sessions fill the context window. Two tools:

```
/compact       # summarizes older turns, keeps recent ones intact
/clear         # wipes conversation, keeps CLAUDE.md
```

Rules of thumb:

- `/compact` when the response starts feeling slow or forgetful
- `/clear` when you switch tasks entirely
- Starting fresh beats fighting a stale context

Why it matters: the model weighs *all* in-context tokens. A 50-turn conversation about three unrelated things confuses it. Compact or clear deliberately.

Worktrees — what they actually are

A **separate working directory** attached to the same `.git` repo, with its own branch checked out — *not* just another branch.

Normal branch switch	Worktree
`git checkout` modifies your *current* dir	`git worktree add` creates a *new* dir
One branch "live" at a time	Multiple branches live in parallel
One working tree	Independent trees, shared `.git`

```
git worktree add ../feat-auth feature/auth   # create
git worktree list                            # list them
git worktree remove ../feat-auth             # delete
```

Worktrees — the simple version

A git repo stores many versions of your project (branches).
Normally, your project folder only shows **one branch at a time** — switching branches swaps which files you see.

A worktree is an extra folder showing a *different* branch — so multiple branches are "visible" on your computer at the same time.

```
WITHOUT WORKTREES                    WITH WORKTREES
_________________                    ______________________________


  ~/myrepo/                            ~/myrepo/          ← showing: main
     └─ files for ONE                  ~/feat-auth/       ← showing: feat/auth
        branch at a time               ~/feat-billing/    ← showing: feat/bill

  Want another branch?                 All three folders exist at once.
  `git checkout` — the                 `cd` between them to switch context.
  files in your folder                 Each has its own files, its own branch,
  get swapped out.                     its own Claude session running.
```

The whole point: edit three branches in parallel, run Claude in each, no file collisions.

Why worktrees + Claude is a superpower

Claude Code stores session state **per directory** (`~/.claude/projects/<path-slug>`). Different worktrees → automatically different, isolated Claude sessions.

What this unlocks:

- **True parallel Claude** — two sessions, two worktrees, zero file collisions
- **Per-worktree permission modes** — one on bypass, one on normal; no cross-contamination
- **Safe sandbox** — let Claude go wild in a throwaway worktree; `git worktree remove` wipes the blast radius
- **Forking a conversation** — stuck? Branch off a worktree, try another path, keep the original alive

Merge-back is just regular git:

```
cd ../feat-auth && git push                # from the feature worktree
cd ../main-repo && git merge feature/auth  # or open a PR
git worktree remove ../feat-auth           # clean up
```

The `ultrathink` keyword

Drop these keywords into any prompt to expand the reasoning budget:

Keyword	Relative effort
`think`	small bump
`think hard`	larger
`think harder` / `megathink`	larger still
`ultrathink`	maximum

Example:

```
ultrathink — our webhook deliveries are dropping ~2% at peak load.
Here's the queue code [@src/queue.ts]. What's the most likely root cause?
```

Why it matters: for debugging, architecture, or subtle correctness questions, extra reasoning pays for itself. Don't use reflexively — it costs tokens.

`AskUserQuestion` — structured decisions

Claude can pause mid-task to ask you 1–4 multiple-choice questions. You've been using it implicitly; you can also prompt for it:

```
I'm about to start the refactor, but two approaches are viable.
Ask me to choose between them with AskUserQuestion before proceeding.
```

Why it matters: structured choices beat free-text Q&A. The UI renders them as clickable options, you can add notes, and the answer is unambiguous in context.

Permission modes — the full table

The UI label (what you see in Claude) and the config name (what you write in `settings.json` / `--permission-mode`) are different. Both exist:

UI label	Config name	Behavior
Normal	`default`	Ask before each tool call
Auto-accept edits	`acceptEdits`	Auto-approve `Read` / `Edit` / `Write`; still ask for `Bash` etc.
Plan	`plan`	Read-only — plans but doesn't edit or run state-changing commands
Bypass permissions	`bypassPermissions`	Skip *every* prompt — sandbox only

`Shift+Tab` cycles the first three. Bypass is only via the flag: `--dangerously-skip-permissions`. Auto-accept applies *inside* turns — you submit a prompt, Claude's edits land without asking.

Configuring permissions in `settings.json`

```
{
  "permissions": {
    "allow": ["Read", "Edit", "Bash(npm test:*)", "Bash(git status:*)"],
    "deny":  ["Bash(rm -rf*)", "Bash(curl:*)", "Bash(sudo:*)"],
    "ask":   ["Bash(git commit:*)", "Bash(git push:*)"]
  }
}
```

Precedence: `deny` > `ask` > `allow`. Glob matching on the tool arg (`:*` matches any args).

Easiest way to write this: have Claude do it for you.

```
Review my shell history + this repo's stack. Draft a `.claude/settings.json`
permissions block that: allows routine dev commands, denies destructive ones,
asks before state-changing git. Explain each line.
```

See `cheatsheets/permissions.md` for a starter template.

Permission patterns that work

- **Read-only by default, write with approval** — allow `Read` / `Grep` / `Glob`; `ask` on `Edit` / `Write`.
- **Pre-approve commands you run all day** — add your project's actual test, build, and lint commands to `allow`. The right set is **the exact commands your repo uses**, not a generic list.
- **Deny the dangerous primitives outright** — `rm -rf`, `curl` / `wget` (exfiltration), `sudo`, `git push --force`, `git reset --hard`.
- **Ask on state-changing git** — `git commit`, `git push`, `git tag`. Cheap to approve; catches mistakes.

How to tune: run for a day, watch which prompts feel like friction, add those to `allow`. Watch which felt scary, add those to `ask` or `deny`.

Bypass permissions — `--dangerously-skip-permissions`

Approves **every** tool call without asking. Fast, dangerous.

Real risks: `rm -rf` · credential exfiltration via prompt injection (`curl` your `.env` to an attacker) · broken git state · production DB if creds are on the host.

Where it's OK: throwaway worktree on a feature branch · fresh scratch repo with nothing of value · short loops where you diff every change.

Where it's NOT: your dev laptop on `main` against a real project · CI with prod secrets · any shared environment.

Honest default: don't reach for bypass unless you've thought through the blast radius. A tight `allow` / `deny` / `ask` config in `settings.json` is almost always the better answer.

4 · Extensibility primitives

Skills, subagents, hooks — when to use each and how they compose

The decision matrix

You want...	Reach for
A reusable prompt/procedure invoked on demand (`/my-thing`)	**Skill**
An isolated sub-Claude with focused tools & context	**Subagent**
Automatic behavior on an event (pre/post tool use, session start)	**Hook**

Rule of thumb:

- Skill = *you* trigger it, deterministic procedure
- Subagent = *Claude* delegates to it, specialized work with its own context budget
- Hook = *the system* runs it, no model involved

Skill anatomy

`~/.claude/skills/<name>/SKILL.md` — YAML frontmatter + markdown body:

```
---
name: <slug>                # becomes the /slash-command
description: <one line>     # tells Claude when to use + user what it does
user-invocable: true        # appears in /slash menu
allowed-tools: <list>       # tight tool budget (least privilege)
argument-hint: "[args]"     # placeholder shown to user
---

# Skill title

<markdown body = the system prompt appended when invoked>

Write it as instructions to Claude: steps, rules, examples, constraints.
```

Invoke: `/<name>`. Also auto-suggested when the user's prompt matches `description`.

We'll walk through a real one (`/tutor`) in §5.

Subagent anatomy

`~/.claude/agents/<name>.md` — same frontmatter-plus-body shape as a skill, different dispatch:

```
---
name: <slug>                # the subagent identity
description: <one line>     # when the parent Claude should delegate to this
tools: <list>               # tool budget — often tighter than main Claude
---

<system prompt for this subagent>

Its role, what to do when invoked, what format to return.
```

Parent Claude delegates via the `Agent` tool. The subagent runs in its **own context window**; only its final summary comes back.

Why it matters: keeps the main conversation lean. Heavy research, deep reviews, or isolated tasks run in a subagent — parent context stays focused.

Hook anatomy

Configured in `settings.json` — shell commands fired on events. No model involvement:

```
{
  "hooks": {
    "<EventName>": [{
      "matcher": "<tool-or-pattern>",
      "hooks": [{ "type": "command", "command": "<shell>" }]
    }]
  }
}
```

Events you can hook: `PreToolUse`, `PostToolUse`, `UserPromptSubmit`, `Stop`, `SubagentStop`, `SessionStart`.

Why it matters: hooks are deterministic — "every time X, do Y". Memory and preferences can be ignored by the model; hooks are system-executed.

▶️ Hook demo — blocking `git push`

Live in `lab-app/.claude/settings.json`:

```
"hooks": {
  "PreToolUse": [{
    "matcher": "Bash",
    "hooks": [{
      "type": "command",
      "if": "Bash(git push:*)",
      "command": "printf '{\"hookSpecificOutput\":{\"hookEventName\":\"PreToolUse\",\"permissionDecision\":\"deny\",\"permissionDecisionReason\":\"🪝 Blocked by demo policy\"}}'"
    }]
  }]
}
```

Setup: paste into `<repo>/.claude/settings.json` → trust the project on next `claude` launch → `/hooks` confirms it loaded.

How it fires: before every `Bash` call, Claude Code pipes a JSON payload to the hook's stdin. `if:` pre-filters by permission-rule pattern — non-matching commands skip the hook entirely. On match, the command prints stdout JSON with `permissionDecision: "deny"`; Claude Code surfaces the `permissionDecisionReason` as a red error block in the transcript. The command never runs.

▶️ **DEMO** · Ask Claude: *"push this to main."* Claude tries `git push`, hook denies, Claude reads the reason from the transcript and reports back — full agent-loop visible on screen.

When to reach for which — worked examples

Scenario	Pick
"Summarize my PRs every Monday"	**Scheduled headless + skill**
"Run eslint before every Edit"	**Hook (PreToolUse on Edit)**
"I want a `/onboard-repo` I can invoke"	**Skill**
"Delegate deep research so my main context stays clean"	**Subagent**
"Log every Bash command Claude runs"	**Hook (PreToolUse on Bash)**
"Interactive Socratic debugging mentor"	**Skill** (see `/tutor` next)

5 · Live build walkthrough

Deploy a real `/tutor` skill, backed by a research subagent, end-to-end

Meet `/tutor`

A real, 285-line skill that turns debugging sessions and code reviews into deep learning moments.

What makes it a good teaching example:

- Non-trivial — 7 invocation modes, argument parsing, doc persistence
- Uses **every** frontmatter field you'd care about
- Uses a **5-layer teaching framework** — the skill body is a prompt-as-code artifact
- Persists results — each lesson is appended to `~/.claude/skills/tutor/docs/<domain>.md`

Full source in the `bonus-skills/` folder of this repo. Drop it into your own `~/.claude/skills/tutor/` to use it.

`/tutor` — reading the frontmatter

```
name: tutor
description: Principal engineer mentor. Context-aware teaching with persistent
  lesson docs. Use when user says "tutor" or wants the "why" behind code.
user-invocable: true
allowed-tools: Bash, Read, Write, Edit, Glob, Grep, WebSearch
argument-hint: "[topic] [--socratic] [--deep] [--review] [--debug] [--progress]"
```

Design choices to steal:

- `description` is **specific** — names trigger words ("tutor", "why"), so Claude auto-suggests it at the right moments.
- `allowed-tools` is **tight** — no `Bash(rm:*)`, no destructive ops. Least privilege.
- `argument-hint` shows flags — users discover the full API without reading the body.

`/tutor` — the body design

The body *is* the system prompt appended when invoked. Four design choices worth stealing:

1. **Flags compose** — `/tutor --socratic --debug "topic"`. One skill + flags beats 7 skills.
2. **Named framework (5 layers)** — Situation → Mechanism → Principle → Adjacent → Pattern Recognition. Predictable shape; easy for the model to obey.
3. **Default vs `--deep`** — ships layers 1–3, offers to expand. Respects time; opt-in depth.
4. **Persistence** — writes lessons to `~/.claude/skills/tutor/docs/<domain>.md` + updates `index.md`. Compounds into a personal knowledge base.

The 5-layer framework (why it works)

```
Layer 1 — Situation     [ground in the concrete problem from context]
Layer 2 — Mechanism     [how it works, Mermaid diagram if useful]
Layer 3 — Principle     [name the rule, explain why it exists]
Layer 4 — Adjacent      [2-3 related concepts to round out the mental model]
Layer 5 — Pattern       ["when you see X, check for Y, because Z"]
```

Why layered: teaching collapses without structure. Without layer 1, lessons feel abstract. Without layer 3, they feel like trivia. Without layer 5, they don't *generalize* — the learner solves this bug but can't spot the class of bug.

The takeaway for YOUR skills: give your skill a *named framework* its body enforces. A skill without structure is just a long prompt; a skill with structure is a reliable tool.

Persistence pattern (steal this)

Every non-review lesson, `/tutor`:

1. **Classifies** the domain (`python`, `databases`, `architecture`, ...)
2. **Reads** the existing topic file if it exists
3. **Appends** a new lesson (or adds an example to an existing one)
4. **Updates** `docs/index.md` with the lesson title + date
5. **Cross-references** related lessons in other domains

Result: a growing, self-organizing personal knowledge base. Fully markdown. Renders in GitHub/Obsidian/VS Code.

Apply this to: `/commit-ready` (log commit patterns), `/incident` (persist RCAs), `/onboard-repo` (build a growing project guide). Any skill whose output *accumulates* should persist.

▶️ Build YOUR Claude Code knowledgebase

Use `/tutor` on concepts from today — each invocation persists a lesson:

```
/tutor --socratic "how CLAUDE.md hierarchy and precedence work"
/tutor --deep "permission modes and when to use each"
/tutor "the difference between skills, subagents, and hooks"
/tutor --progress
```

Result: `~/.claude/skills/tutor/docs/claude-code.md` fills in over time. A week from now, `--progress` shows a growing map — not just Claude Code, but your whole engineering stack.

▶️ **DEMO** · In `lab-app/`, run `/tutor --research "CSS color contrast ratios for accessibility"`. Watch the `tutor-researcher` subagent gather sources, `/tutor` cite them inline, and a lesson persist to `~/.claude/skills/tutor/docs/`.

Composition — skill + subagent + persistence

Extending `/tutor` into a full pipeline with `/tutor --research "topic"`:

```
┌──────────────────────┐
│ /tutor --research    │
└──────────────────────┘
          │
          ▼
┌──────────────────────┐   Subagent — own context,
│ tutor-researcher     │   scoped tools, returns
│ (sources + prior     │   structured findings
│  art + citations)    │
└──────────────────────┘
          │
          ▼
┌──────────────────────┐   Skill body — synthesizes
│ /tutor teaches with  │   the 5-layer lesson WITH
│ inline citations     │   inline refs + bibliography
└──────────────────────┘
          │
          ▼
┌──────────────────────────────┐   Appends lesson to
│ ~/.claude/skills/tutor/docs/ │   the domain file,
│   <domain>.md                │   updates docs/index.md
│   index.md                   │
└──────────────────────────────┘
```

The research subagent (anatomy)

`~/.claude/agents/tutor-researcher.md`:

```
---
name: tutor-researcher
description: Deep-research companion for /tutor. Gathers sources + surfaces
  prior lessons; returns structured findings with citations.
tools: WebSearch, WebFetch, Read, Grep, Glob
---
```

Why a subagent and not just more prompt? Subagents run in their own **isolated context window** with a **scoped tool budget**. Web-search noise, half-read pages, and dead-end queries stay out of the main conversation — only the distilled findings come back. Reach for one when a task has a clear input/output contract, benefits from depth, and would otherwise pollute context you need to keep lean.

6 · Where to go next

Resources

- **Docs:** docs.claude.com/claude-code
- **Claude Code repo:** github.com/anthropics/claude-code
- **This talk + lab:** github.com/debozkurt/claude-intro — `lab-app/` · `cheatsheets/` · `bonus-skills/`

What to build first — in order

1. Write a `CLAUDE.md` for a repo you know well. Watch suggestions sharpen.
2. Lock down `settings.json` permissions. Notice prompts drop.
3. Ship one skill — start with `/commit-ready`.
4. Ship one hook — `PreToolUse` on `Bash` that logs commands.
5. Add a subagent when a task wants its own context budget.

Thanks!

Let's stay in touch

Repo: github.com/debozkurt/claude-intro

LinkedIn: linkedin.com/in/debozkurt

Prompt Engineers: promptengineers.ai · meetup.com/plano-prompt-engineers

Email: debozkurt@gmail.com